Contents

Parametric polymorphism provides a language-independent abstraction, its implementation varies considerably among programming languages and compiler architectures.

The principal design choice concerns how generic types are represented after compilation and when concrete type instantiations are produced.

These implementation strategies influence several aspects of the execution model, including runtime representation, reflection capabilities, performance, and binary size.

The following dimensions constitute the primary axes along which implementations of parametric polymorphism are typically compared.

0.1 Instantiation Time

A first distinguishing characteristic is the stage at which generic definitions are instantiated.

Some implementations perform instantiation entirely during compilation.

In this approach, each concrete use of a generic function or data structure is expanded into a specialized implementation before executable code is generated.

Since all specializations already exist in the final binary, runtime execution incurs no additional generic instantiation cost.

Other implementations compile generic definitions only once.

Generic type parameters are verified during type checking and subsequently removed or replaced by a common representation.

Consequently, no additional executable code is generated for different type arguments, and all instantiations share the same compiled implementation.

0.2 Runtime Representation

Implementations also differ in how generic types are represented after compilation.

Type erasure removes generic type parameters, producing a homogeneous representation where different instantiations share the same executable code.

The runtime system therefore treats different generic instantiations as equivalent representations.

Conversely, *reified generics* preserve concrete type information by generating distinct specialized code for each instantiated type.

Each generic instantiation may therefore possess its own runtime representation, memory layout, and executable instructions.

0.3 Reflection and Runtime Type Information

The availability of generic type information at runtime depends directly on the chosen representation strategy.

When type erasure is employed, generic parameters are generally unavailable after compilation.

Reflection mechanisms therefore provide limited access to instantiated generic types because most type information has been discarded.

Conversely, implementations based on reified generics preserve concrete type information throughout compilation.

Runtime reflection or runtime type identification (RTTI) may therefore recover the actual instantiated types, enabling operations such as generic type inspection, specialized serialization, and runtime metadata analysis.

0.4 Type Safety

Both implementation strategies preserve static type safety, although they achieve this objective differently.

In implementations based on type erasure, generic correctness is verified entirely during compilation.

Although type parameters disappear from the generated executable, compiler-inserted casts preserve the guarantees established by the static type system.

Implementations based on reified generics retain complete type information throughout compilation and generate type-specific implementations directly.

Since each specialization corresponds to a concrete type, correctness is preserved without requiring erased representations or implicit runtime casts.

0.5 Performance

The implementation strategy significantly influences both compilation and execution performance.

Homogeneous implementations generally produce smaller binaries and reduce compilation time because generic definitions are compiled only once.

However, the shared representation may require additional indirections, boxing, or runtime casts, limiting optimization opportunities.

Heterogeneous implementations incur greater compilation cost and larger executables because every instantiated type generates a distinct implementation.

Nevertheless, the compiler can aggressively optimize each specialization, exploiting concrete type information for improved instruction selection, memory layout, inlining, and register allocation.

Consequently, runtime performance often approaches that of manually specialized code.

0.6 Summary

The implementation of parametric polymorphism is primarily characterized by five orthogonal dimensions:

1. the stage at which generic instantiation occurs;
2. the runtime representation of generic types;

3. the availability of runtime type information and reflection;

4. the trade-off between compilation cost, binary size, and execution performance;

5. the mechanisms through which static type safety is preserved.

These dimensions form the basis for comparing homogeneous implementations based on type erasure with heterogeneous implementations based on reification or monomorphization.

Chapter 1

C++ Templates

1.1 Instantiation Time

C++ implements parametric polymorphism through the *template* mechanism, which adopts a purely compile-time instantiation model.

Unlike languages that compile a generic definition only once, a C++ template is not itself an executable entity. Instead, it represents a parameterized program definition that serves as a blueprint from which the compiler generates independent concrete implementations.

From the perspective of instantiation time, the defining characteristic of C++ is that executable code is generated only after the compiler knows the complete set of template arguments.

Consequently, all template instantiations are resolved entirely during static compilation, and no template instantiation mechanism exists during program execution.

The complete instantiation process can be understood by following the template through the compilation pipeline.

1.1.1 Template Definition

When the compiler encounters a template definition, it first performs the ordinary front-end compilation stages, including lexical analysis, parsing, and semantic analysis that can be performed without knowledge of future template arguments.

For example:

```
template<typename T>
T get_max(T a, T b)
{
    return (a > b) ? a : b;
}
```

The compiler parses the template syntax, builds the corresponding Abstract Syntax Tree (AST), checks syntactic correctness, and stores an internal parameterized representation.

At this stage, the compiler does not generate executable machine code.

The reason is straightforward: the compiler still ignores the concrete meaning of the template parameter `T`, and therefore cannot determine the exact operations, object layout, calling convention, or generated instructions required by the final implementation.

Consequently, the template definition remains a generic blueprint waiting for future instantiation.

1.1.2 Instantiation Trigger

The instantiation process begins only when the compiler encounters a concrete use of the template, either through an explicit specification of the template argument or through implicit type deduction.

Example.

Consider the following template invocations:

```
int i = get_max(3, 7);
double d = get_max<double>(2.5, 4.8);
```

The first invocation relies on template argument deduction:

$$T = int$$

whereas the second invocation explicitly specifies the template argument:

$$T = double$$

Therefore, the compiler generates two different template instantiations: `get_max<int>` and `get_max<double>`

Each unique set of template arguments constitutes a distinct template instantiation. Instantiation may occur through:

- `implicit instantiation`, automatically performed whenever a specialization is required by the program;
- `explicit instantiation`, requested directly by the programmer through an explicit instantiation declaration or definition;
- `explicit specialization`, where an entirely different implementation is provided for a specific combination of template arguments.

Regardless of how instantiation is initiated, executable code generation begins only after the complete set of template arguments is known.

1.1.3 Template Argument Substitution

Once instantiation has been triggered, the compiler performs template argument substitution.

During this phase, every formal template parameter is replaced by its corresponding concrete argument.

From previous example.

$$T \to int$$

produces the specialization:

```
int get_max(int a, int b)
{
    return (a > b) ? a : b;
}
```

whereas:

$$T \to double$$

produces:

```
double get_max<double>(double a, double b)
{
    return (a > b) ? a : b;
}
```

Conceptually, the compiler now treats each specialization as if it had been written explicitly by the programmer.

From this point onward, the specialization behaves as an ordinary non-template function.

1.1.4 Semantic Analysis of the Specialization

Instantiation does not immediately produce executable code.

Instead, each generated specialization undergoes the normal semantic analysis performed for ordinary C++ program entities.

Depending on the specialization, the compiler performs activities such as: type checking, overload resolution, name lookup, constraint verification for constrained templates, constant expression evaluation, instantiation of nested templates and verification of language rules required by the selected template arguments.

This step is fundamental because many semantic properties cannot be verified before template parameters have been substituted.

Consequently, a template definition that is syntactically correct may still produce compilation errors if one of its concrete instantiations violates the requirements imposed by the template body.

1.1.5 Intermediate Representation and Code Generation

Once semantic analysis has successfully completed, the specialization enters the compiler back-end.

The compiler generates an Intermediate Representation (IR) specifically for the instantiated specialization.

Subsequently, the specialization undergoes the standard optimization pipeline, which may include: constant propagation, dead-code elimination, function inlining, loop optimizations, register allocation, instruction selection and target-specific optimizations.

Finally, the compiler emits native machine code corresponding exclusively to that specialization.

Importantly, this process is repeated independently for every distinct template instantiation.

Consequently, each specialization owns an independent executable implementation. In fact, the compiler does not produce a single shared executable representation capable of serving multiple template arguments.

1.1.6 Completion of the Instantiation Process

The instantiation process terminates entirely before linking.

By the time object files are generated, every template specialization required by the translation unit already exists as ordinary compiled code.

After linking, the final executable therefore contains every specialization required by the program.

From the perspective of executable code generation, the lifecycle of a C++ template can be summarized as follows:

1. The compiler parses the template definition.
2. The template is stored as a parameterized internal representation.
3. A concrete template use triggers instantiation.
4. Template parameters are substituted with concrete arguments.
5. The generated specialization undergoes complete semantic analysis.
6. The compiler produces a specialization-specific intermediate representation.
7. Optimization passes are applied.
8. Native machine code is emitted.
9. The specialization is included in the final executable before program execution begins.

Therefore, the defining characteristic of C++ parametric polymorphism is that the entire instantiation process is completed during static compilation.

When execution begins, no template instantiation mechanism remains active.

The runtime simply executes machine code that has already been generated for every required specialization during compilation.

1.2 Runtime Representation

C++ templates are implemented according to a translation model in which generic source-level abstractions are transformed into concrete program entities before execution. A template declaration itself does not correspond to a runtime object, runtime function, or executable structure. Instead, it defines a family of possible entities that are materialized only when the compiler generates a specific specialization.

The runtime representation of a C++ template specialization is therefore equivalent to the representation of an ordinary C++ function, class, or object. After compilation and linking, the executable contains native machine instructions and data structures associated with each instantiated specialization. The execution environment, including the operating system loader and the processor, has no knowledge of the original template declaration, template parameters, or instantiation process. It only observes ordinary executable artifacts such as functions, symbols, and memory objects.

A template can be conceptually modeled as a compile-time transformation:

$$T : (A_1, A_2, ..., A_n) \rightarrow P$$

where T represents a template definition, $(A_1, A_2, ..., A_n)$ are template arguments, and P is a concrete program entity generated from the template.

For example:

$$T < int > \rightarrow P_{int}$$

$$T < double > \rightarrow P_{double}$$

The generated entities are independent concrete implementations. During execution, the program does not invoke a generic representation of T; instead, it executes the machine code generated for each concrete specialization.

1.2.1 Runtime Form of Template Specializations

The C++ compilation model transforms template specializations into ordinary binary-level entities. Consider the following function template:

```
template <typename T>
T add(T a, T b)
{
    return a + b;
}
```

When the program requires multiple specializations:

```
add<int>(1,2);
add<double>(1.0,2.0);
```

the compiler produces separate functions:

$$add < int > (int, int)$$

and

$$add < double > (double, double)$$

At the binary level, these functions are represented as independent machine code regions. Each specialization has its own instructions, symbol identity, and executable address. The runtime system does not contain a generic function body with a runtime type parameter. Instead, it contains the concrete implementations produced during translation.

Once the executable is loaded into memory, the operating system maps the binary sections containing executable instructions and data. Control flow transfers directly to the generated machine-code addresses. The processor executes instructions without any awareness that those instructions were originally produced from a template definition.

Therefore, the runtime representation of:

$$Vector < int >$$

and:

$$Vector < double >$$

is not a single generic runtime object:

$$Vector < T >$$

but two independent concrete program entities:

$$Vector < int >$$

and:

$$Vector < double >$$

Each specialization exists as a normal C++ type produced before execution.

1.2.2 Monomorphization

C++ uses heterogeneous translation, commonly described as *monomorphization*, to implement templates. This mechanism transforms generic template definitions into concrete specialized entities for each set of template arguments used by the program.

The essential property of monomorphization is that every specialization receives its own native representation. The compiler substitutes the template arguments into the template definition and generates a concrete function or class implementation.

For example:

```
template <typename T>
struct Container
{
    T value;
};

Container<int> a;
Container<float> b;
```

creates two independent specializations:

$$Container < int >$$

and

$$Container < float >$$

The executable contains the concrete representations of these entities. There is no runtime object representing:

$$Container < T >$$

because the generic abstraction has already been resolved during translation.

For function templates, each specialization produces a distinct machine-code artifact:

$$Compiler(T < int >) = MachineCode_{int}$$

$$Compiler(T < double >) = MachineCode_{double}$$

These generated artifacts behave exactly like functions written explicitly for those types. After compilation, functions such as `sort<int>()` and `sort<double>()` are ordinary executable functions with independent entry points, invoked through normal machine-level control-flow instructions.

1.2.3 Name Mangling

Compilers encode template arguments into symbol names through *name mangling*. This allows different specializations to coexist in object files and allows the linker to distinguish between entities such as `function<int>()` and `function<double>()`.

Name mangling is a compiler and linker mechanism used during binary generation; it is not runtime metadata. Once linking is completed, the CPU does not interpret mangled names or template information. The executable address associated with a symbol identifies machine code, and the processor executes this code without understanding the source-level template relationship.

1.2.4 Final Executable Representation

After linking, the final executable represents a collection of concrete compiled entities:

$$Executable = \{Code(P_1), Code(P_2), ..., Code(P_n)\}$$

where each P_i corresponds to a particular template specialization. The executable contains:

- machine instructions for instantiated functions;
- symbols identifying generated entities;
- relocation information required during linking;
- optional runtime metadata produced by other C++ mechanisms (see Section 1.5).

These artifacts are not different from those produced by non-template C++ code.

During execution, the processor does not interpret template parameters. A machine instruction such as a function call transfers execution to a specific address. The target address corresponds to a compiled function, not to a generic template definition. The relationship between a generated function and its original template source exists only within compiler-generated information such as debugging information or symbol naming conventions; it is not part of the runtime execution model.

1.2.5 Absence of Generic Metadata

A defining characteristic of the C++ template implementation model is the absence of runtime generic metadata. After template instantiation has occurred, the executable does not contain information describing the original template declaration or the possible template arguments.

The runtime environment does not know that a particular function originated from:

```
template<typename T>
void process(T value);
```

Instead, it only observes a concrete function generated from a specific instantiation, such as `process<int>()` or `process<double>()`.

The following concepts do not exist as runtime entities:

- template declarations;
- template parameter lists;
- generic type variables;
- mechanisms for creating new template specializations dynamically.

The execution environment only manipulates the final native representation. The limited runtime type identification that C++ does provide is discussed in Section 1.5.

1.3 Performance Characteristics

The preservation of complete type information through template instantiation has direct consequences for execution performance. Although *reification* describes the conceptual property of retaining type information throughout compilation, the performance characteristics of C++ templates derive from the concrete implementation mechanism of template monomorphization described in Section 1.2.2.

During compilation, every template instantiation is transformed into an independent specialization operating on a specific concrete type. Therefore, the generated executable contains ordinary statically typed functions and classes rather than a generic runtime representation. The compiler does not generate a universal implementation capable of handling arbitrary types; instead, it produces optimized versions tailored to each instantiated type.

For example, the following template function:

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

when instantiated with different types, generates specialized functions equivalent to:

```
int max(int a, int b) {
    return (a > b) ? a : b;
}

double max(double a, double b) {
    return (a > b) ? a : b;
}
```

The resulting machine code operates directly on concrete data types, without requiring runtime mechanisms for interpreting or resolving the generic type parameter.

1.3.1 Static Dispatch and Elimination of Runtime Overhead

One of the main performance advantages introduced by template monomorphization is static dispatch. Since the concrete type of every template parameter is known during compilation, function calls can be resolved before execution. This eliminates the runtime indirections required by dynamic dispatch mechanisms and allows the compiler to directly generate the target function call.

For example:

```
template<typename T>
void process(T& object) {
    object.compute();
}
```

with a concrete type:

```
Matrix m;
process(m);
```

allows the compiler to resolve the call directly to `Matrix::compute()`. The absence of runtime dispatch reduces execution overhead and exposes additional opportunities for optimization.

Another direct consequence of template specialization is the elimination of boxing and unboxing operations. Template-generated code operates directly on the native representation of objects and primitive values, avoiding additional conversions or wrapper objects.

For example:

```
std::vector<int> values;
values.push_back(10);
```

stores integer values using their native representation. The generated implementation does not require conversion of the integer into a generic object representation before storage or computation.

1.3.2 Compiler Optimization Opportunities

Because every template specialization is fully typed, optimizing compilers can apply the same optimization techniques used for manually written specialized code. The availability of concrete types, sizes, and operations enables more aggressive analysis and transformation of the generated machine code.

A fundamental optimization enabled by template specialization is function inlining. Since both the function implementation and the argument types are known at compile time, the compiler can replace a function call with the corresponding function body.

For example:

```
template<typename T>
inline T square(T x) {
    return x * x;
}

int result = square(5);
```

can be transformed into code equivalent to:

```
int result = 5 * 5;
```

removing the function call overhead and enabling further optimizations, such as:

- constant propagation;
- constant folding;
- dead-code elimination;

- loop optimization;
- strength reduction.

Template parameters can also represent compile-time constants. When values such as array sizes or loop bounds are known during compilation, the compiler can simplify control flow and remove unnecessary runtime operations.

For example:

```
template<int N>
int sum(const int* data) {
    int result = 0;

    for(int i = 0; i < N; i++)
        result += data[i];

    return result;
}
```

allows the compiler to know the exact iteration count. Consequently, loop unrolling can be applied, replacing repeated loop iterations with explicit instructions and reducing loop-control overhead.

Similarly, compile-time specialization facilitates branch elimination through features such as `if constexpr`:

```
template<typename T>
void process(T value) {

    if constexpr(std::is_integral_v<T>)
        integer_algorithm(value);
    else
        generic_algorithm(value);
}
```

Only the branch corresponding to the selected type is generated in the final code, eliminating unnecessary runtime condition checks.

The availability of concrete types and memory layouts also improves the ability of compilers to perform automatic vectorization. For numerical templates, the compiler can recognize regular memory access patterns and generate SIMD instructions when applicable.

```
template<typename T>
void add(const T* a,
         const T* b,
         T* result,
         int n)
{
    for(int i = 0; i < n; i++)
        result[i] = a[i] + b[i];
}
```

When instantiated with numerical types, the generated code can be optimized using vector instructions, allowing multiple operations to be executed simultaneously.

1.3.3 Zero-Overhead Abstraction and Trade-offs

The combination of static dispatch, direct data representation, and compiler optimization enables C++ templates to follow the *zero-overhead abstraction* principle. According to this principle, abstractions introduced at the source level should not impose additional runtime costs compared with an equivalent manually specialized implementation. Template-based abstractions can therefore provide generic programming capabilities while maintaining execution efficiency comparable to explicitly written type-specific code.

The main performance limitation associated with template monomorphization is related to the expansion of generated code. Since every specialization produces an independent implementation, extensive template usage may increase executable size. A larger instruction footprint can negatively affect instruction cache locality, increasing instruction cache misses in performance-critical applications.

This effect does not originate from additional runtime computation introduced by templates, but from the interaction between generated specialized code and the processor memory hierarchy. Therefore, template performance is generally dominated by the benefits of compile-time specialization, while instruction cache behavior represents the main architectural factor capable of reducing these advantages.

1.4 Type Safety

Static type safety represents one of the fundamental objectives of parametric polymorphism. Its purpose is to guarantee that every operation performed on a value is compatible with its statically known type before program execution. In this way, incorrect type interactions are detected during compilation rather than being deferred to runtime.

Different implementations of parametric polymorphism achieve static type safety through different mechanisms, as introduced in the overview of implementation strategies. C++ templates adopt a reified implementation strategy based on the template instantiation and monomorphization process described in Section 1.2.2: type parameters are never erased, and every instantiation is checked against a concrete, fully specialized implementation.

From a type system perspective, reification allows the compiler to retain complete information about the actual types involved in every specialization. Unlike erased representations, where the original type parameter is no longer available during execution, C++ templates preserve the relationship between the generic definition and the concrete types used to instantiate it. Consequently, all type-dependent operations are checked directly against the substituted types.

1.4.1 Verification of Template Correctness

The verification of template correctness occurs through the ordinary C++ static type system. After template argument substitution, the compiler performs semantic analysis on the generated specialization, checking that all expressions, operators, conversions, and function calls are valid for the concrete type. If a required operation is not supported by the instantiated type, the specialization is considered ill-formed and the program is rejected during compilation.

This mechanism differs fundamentally from implementations based on type erasure because C++ templates never require the compiler to recover discarded type information. Since each specialization is generated from a known concrete type, operations are resolved statically according to the rules of the language type system. Therefore, no implicit runtime casts are required to restore the original generic type information.

1.4.2 Example: Type Checking Across Specializations

The following example illustrates how C++ templates preserve static type safety through concrete specialization.

```
#include <string>

template <typename T>
T maximum(const T& a, const T& b)
{
    return (a < b) ? b : a;
}

int main()
{
    int x = maximum(3, 7);

    double y = maximum(3.5, 8.1);

    // Compilation error:
    // no valid specialization exists because
    // the arguments do not have the same type.

    // auto z = maximum(3, std::string("hello"));
}
```

The function template `maximum` describes a generic operation that can be applied to any type satisfying the required semantic constraints. When the compiler encounters the expression `maximum(3, 7)`, template argument deduction determines that the parameter `T` corresponds to the type `int`. The compiler then creates a specialization equivalent to:

```
int maximum(const int& a, const int& b);
```

Similarly, the invocation `maximum(3.5, 8.1)` produces a specialization where `T = double`, and the generated function operates exclusively on values of type `double`.

Each specialization is independently checked by the compiler. The comparison operator `<` must exist for the instantiated type, the return expression must be compatible with the declared return type, and all parameter bindings must satisfy the requirements imposed by the C++ type system.

The invalid invocation `maximum(3, std::string("hello"))` cannot produce a valid specialization because template argument deduction cannot identify a single type for `T`. The compiler therefore rejects the program before executable code is generated. No runtime type inspection, dynamic cast, or conversion mechanism is involved, because the type inconsistency is detected entirely during static analysis.

1.4.3 Type Safety Across Instantiations

A further consequence of template reification is that type checking is performed with respect to each concrete template instantiation. A template definition represents a family of possible programs, while every generated specialization represents a fully specified program whose types are known. Therefore, correctness is established separately for every instantiated version rather than through a single erased generic representation.

This property means that template abstraction does not weaken the C++ static type system. Instead, templates delay the determination of concrete types until instantiation, after which normal compile-time type checking is applied to the resulting program. Every successfully generated specialization is consequently an ordinary statically typed C++ entity, whose correctness has been verified before execution.

From a type-theoretic perspective, C++ templates achieve static type safety through reification: generic definitions preserve type information, template arguments are substituted with concrete types, and the resulting specializations are validated according to the complete rules of the language type system. This approach guarantees generic correctness without requiring erased representations or compiler-generated runtime casts.

1.5 Reflection and Runtime Type Information

As introduced in the overview of implementation strategies, the availability of generic type information at runtime depends directly on whether the implementation follows a type-erasure or a reified model. C++ templates follow the reified model described in Section 1.2: template parameters are not represented as a single runtime generic entity, and monomorphization preserves the identity of the concrete types used during compilation. However, compile-time type preservation should not be confused with runtime reflection. After compilation, the C++ execution environment does not generally provide a universal mechanism capable of enumerating arbitrary template parameters, inspecting source-level declarations, or dynamically querying all properties of instantiated template entities. What C++ does provide is a restricted mechanism, Run-Time Type Information (RTTI), which allows limited runtime identification of concrete types.

1.5.1 The `typeid` Operator and `std::type_info`

The primary standard facilities for RTTI are the operators `typeid` and `dynamic_cast`, together with the `std::type_info` class.

The `typeid` operator obtains runtime type information associated with an expression or a type. When applied to an instantiated template class, it can identify the concrete specialization generated by the compiler. For example:

```
template <typename T>
class Wrapper {
};

Wrapper<int> a;
Wrapper<double> b;

std::cout << typeid(a).name();
std::cout << typeid(b).name();
```

The two objects correspond to different C++ types, namely `Wrapper<int>` and `Wrapper<double>`. Consequently, the RTTI system treats them as distinct runtime types whenever RTTI metadata is generated for these entities. The resulting `std::type_info` objects represent the runtime identity of the instantiated classes.

The information exposed by `std::type_info` is intentionally limited. It allows comparison of runtime type identities through operations such as equality comparison and provides an implementation-defined textual representation through `name()`. It does not provide access to template arguments, class members, inheritance hierarchies, or source-level metadata in the manner of a full reflection system.

1.5.2 `dynamic_cast` and Polymorphic Template Classes

The `dynamic_cast` operator provides another RTTI mechanism for performing safe runtime conversions within polymorphic class hierarchies. A class must contain at least one virtual function for RTTI-based dynamic identification to be available.

Template classes can participate in such hierarchies like ordinary C++ classes. For example:

```
class Base {
    public:
    virtual ~Base() = default;
};

template <typename T>
class Derived : public Base {
};

Base* ptr = new Derived<int>();

Derived<int>* result = dynamic_cast<Derived<int>*>(ptr);
```

During execution, `dynamic_cast` uses runtime type information associated with the object to verify whether the actual object type corresponds to the requested target type. In this example, the cast succeeds because the dynamically stored object is an instance of the concrete template specialization `Derived<int>`.

The same operation would fail for an object instantiated with a different template argument, such as `Derived<double>`, because each template specialization represents a distinct C++ type. Therefore, RTTI preserves the identity of instantiated template classes when those classes are part of a polymorphic hierarchy.

1.5.3 Limits of RTTI as a Reflection Mechanism

C++ templates demonstrate a hybrid relationship between type preservation and runtime identification. The template mechanism preserves concrete type information during compilation through specialization generation, but this information is not automatically exposed through a general runtime reflection interface.

The compiler knows the complete instantiated type during template compilation. It can therefore perform static analysis, select specialized implementations, and generate code based on the concrete template arguments. This information belongs to the compile-time type system and is consumed before program execution.

RTTI, in contrast, represents a restricted runtime mechanism. It operates only on the subset of type information explicitly retained by the C++ object model, primarily for polymorphic objects and explicit type identification through `typeid`. For example, a template class such as:

```
template<typename T>
class Base
{
public:
    virtual ~Base(){}
};

Base<int> object;
```

may generate runtime type information describing `Base<int>`, but it does not generate a runtime descriptor describing the generic definition `Base<T>`, because the generic template abstraction has no runtime existence (Section 1.2).

Consequently, RTTI can distinguish between instantiated template types when the compiler emits the corresponding runtime metadata, but it cannot reconstruct the complete template definition or inspect arbitrary template parameters dynamically. C++ templates therefore preserve generic type information through compile-time monomorphization rather than through runtime reification, and standard RTTI mechanisms provide runtime access to the identity of concrete instantiated types without constituting a complete reflection system.

Chapter 2

Java Generics

2.1 Instantiation Time

Java Generics adopt a compilation strategy based on compile-time instantiation followed by type erasure. During compilation, the Java compiler associates concrete type arguments with generic declarations in order to perform static type checking. However, this association does not lead to the generation of a new specialized class representation.

This design differs fundamentally from the approach adopted by C++ templates. While Java uses instantiation only as a compile-time type verification mechanism, C++ performs template instantiation through monomorphization, generating a separate specialization for every concrete type argument.

2.1.1 The Java Generic Instantiation Model

Consider the following generic declaration:

```
class Box<T> {
    private T value;

    public T get() {
        return value;
    }
}
```

When the compiler encounters:

```
Box<String> box = new Box<String>();
```

the type parameter T is instantiated with the argument `String` for the purpose of compile-time verification. The compiler therefore checks that every operation involving `box` respects the constraint:

$$T = String$$

However, this instantiation does not generate a new runtime class:

$$Box < String >$$

Instead, Java produces a single generic class representation that is shared by all possible instantiations.

2.1.2 Type Erasure Pipeline

After generic type checking, the Java compiler applies type erasure. The type parameter is removed and replaced by its erasure according to the rules defined by the Java Language Specification.

For an unbounded parameter:

$$< T >$$

the erased type becomes:

$$Object$$

Therefore:

```
class Box<T>
```

is transformed into a representation equivalent to:

```
class Box {
    Object value;

    Object get()
}
```

At this point, Java has completed the generic instantiation process. No additional class corresponding to:

$$Box < String >$$

exists in the JVM.

2.1.3 Instantiation with Type Bounds

When a Java generic parameter has a bound:

```
class NumericBox<T extends Number>
```

the compiler verifies that every instantiation satisfies:

$$T <: Number$$

The bound also determines the erasure type. According to the Java Language Specification, the erasure of a type variable corresponds to the erasure of its leftmost bound.

Therefore:

```
NumericBox<Integer>
NumericBox<Double>
```

are both translated into a representation equivalent to:

```
NumericBox
```

with:

```
Number value;
```

2.1.4 Compiler Adaptation After Erasure

Because Java removes generic parameters before execution, the compiler must insert additional bytecode instructions to preserve the semantics established during compile-time instantiation.

For example:

```
String s = box.get();
```

is compiled into a call returning:

```
Object get()
```

followed by an inserted cast:

```
checkcast java/lang/String
```

Across all these mechanisms, the Java instantiation model consistently contrasts with C++ template compilation: whereas C++ monomorphization generates a distinct specialization for every concrete type argument, with type information encoded directly into each generated implementation, Java instantiation is entirely a compile-time verification step that resolves to a single shared runtime representation, requiring the compiler to insert casts and rely on erasure rules precisely because no specialized, type-specific code is produced.

2.2 Runtime Representation

Java Generics are implemented through a *type erasure* model, in which generic type parameters are removed from the executable representation before runtime. The generic abstraction exists only during compilation, where the compiler performs static type checking, verifies type constraints, and generates bytecode compatible with the non-generic JVM execution model.

The fundamental design principle of Java Generics is that different parameterizations of the same generic class share a single runtime representation. For example, the source-level types

$$List < String > \quad \text{and} \quad List < Integer >$$

do not correspond to two different classes inside the JVM. Instead, both are represented by the same runtime class:

$$List$$

The JVM runtime does not maintain separate metadata, object layouts, or method implementations for different generic arguments.

2.2.1 Erasure of Generic Type Parameters

The Java compiler transforms parameterized types by replacing type variables with their erasures. According to the Java Language Specification, the erasure of an unbounded type variable is `Object`, while a bounded type variable is replaced by the erasure of its first bound.

Consider the generic declaration:

```
class Box<T> {
    private T value;

    T get() {
        return value;
    }
}
```

After erasure, the runtime representation is equivalent to:

```
class Box {
    private Object value;

    Object get() {
        return value;
    }
}
```

The JVM therefore executes a representation where the type variable T has been replaced by its erased type. The runtime system has no knowledge that the original source-level declaration used a generic parameter.

Formally, the transformation can be expressed as:

$$Box < T > \rightarrow Box < Object >$$

for an unbounded type variable.

Consequently, the runtime identity of an object is determined only by the erased class:

$$RuntimeType(Box < String >) = RuntimeType(Box < Integer >) = Box$$

The generic parameter is not part of the runtime type identity.

2.2.2 Object Layout and Storage Representation

Because Java Generics do not create specialized runtime classes, generic objects use the same memory representation as ordinary JVM objects.

A generic reference field:

$$T\ value$$

is represented after erasure as:

$$Object\ value$$

The JVM stores a reference to an object rather than a value specialized for the concrete generic type.

For example, an instance of:

$$ArrayList < String >$$

and an instance of:

$$ArrayList < Integer >$$

have identical internal structural representations. The JVM does not allocate a specialized array of strings or integers for these collections. The generic container stores references using the standard JVM reference representation.

This differs from a reified implementation, such as C++ template monomorphization, where each concrete instantiation may generate a distinct object layout and specialized machine code. In Java, the same erased representation is reused for all parameterizations.

2.2.3 Inserted Runtime Casts

Since generic parameters are removed before execution, the compiler inserts runtime casts whenever a value retrieved from a generic structure must be treated as a more specific type.

Example:

```
String s = list.get(0);
```

The erased execution model becomes equivalent to:

```
Object tmp = list.get(0);
String s = (String) tmp;
```

At the bytecode level, the JVM executes a `checkcast` instruction.

The runtime sequence is therefore:

retrieve erased reference → check actual object type → use typed reference

The runtime verification concerns the concrete class of the object, not the generic argument. The JVM checks whether the object is a `String`, but it does not verify the existence of an original `List<String>` instantiation.

2.2.4 Bridge Methods

Type erasure can modify method signatures and break overriding relationships at the JVM level. To preserve polymorphism, the Java compiler generates synthetic *bridge methods*.

Example:

```
class Node<T> {
    T get();
}

class StringNode extends Node<String> {
    String get();
}
```

Before erasure, the overriding relationship is:

$$Node < T > .get() : T$$

overridden by:

$$StringNode.get() : String$$

After erasure, the parent method becomes:

$$get() : Object$$

while the child method remains:

$$get() : String$$

Since the JVM method descriptors differ, the compiler generates a bridge:

```
Object get() {
    return get();
}
```

The bridge method adapts the specialized method to the erased representation, allowing normal JVM method dispatch to operate correctly.

Bridge methods are therefore a direct runtime consequence of the decision to implement generics through erasure rather than through specialized generated types.

2.2.5 Primitive Types and Generic Representation

Java Generics are restricted to reference types because erased generic parameters are represented using reference-compatible types.

A primitive instantiation:

$$List < int >$$

is not permitted.

Instead, the corresponding wrapper type must be used:

$$List < Integer >$$

The runtime representation therefore becomes:

$$int \rightarrow Integer \rightarrow Object$$

The primitive value is converted into an object representation through boxing, and retrieved values require unboxing.

This behavior follows directly from the erased representation: since every generic parameter is mapped to a reference type, the JVM cannot create a specialized generic representation containing primitive values.

2.2.6 Comparison with C++ Templates

C++ Templates are based on a different runtime representation strategy: *monomorphization*. Unlike Java Generics, C++ compilers generate a separate representation for each concrete template instantiation.

This difference explains why Java uses one erased runtime representation:

$$List < T > \rightarrow List < Object >$$

while C++ can generate independent specialized representations:

$$Vector < int >,\ Vector < double >, \ldots$$

The Java approach avoids multiple generated implementations but sacrifices runtime visibility of generic parameters and requires mechanisms such as casts, boxing, and bridge methods. C++ preserves concrete type information in the generated executable by creating specialized versions, but this may increase generated code size.

The central distinction is therefore:

$$\boxed{\text{Java Generics preserve abstraction through erasure}}$$

whereas:

$$\boxed{\text{C++ Templates preserve abstraction through specialization}}$$

For Java, the generic type parameter is a compile-time concept. For C++, template instantiation becomes part of the generated runtime representation.

2.3 Performance Characteristics

The execution performance of generic programming is strongly determined by the translation strategy adopted by the language implementation. Java Generics and C++ templates represent two fundamentally different approaches: Java employs a *type erasure* model, whereas C++ relies on compile-time instantiation and *monomorphization*. These strategies produce different runtime representations and consequently different execution characteristics.

Java Generics are implemented through type erasure, as specified by the Java Language Specification. During compilation, generic type parameters are removed from the generated bytecode and replaced by their upper bounds, typically `Object`. Generic information is preserved only in metadata attributes used by the compiler and runtime tools, but it does not determine the actual machine representation of executed objects.

For example, the generic class:

```
class Container<T> {

    T value;

    T get() {
        return value;
    }
}
```

is translated approximately into:

```
class Container {

    Object value;

    Object get() {
        return value;
    }
}
```

Consequently, the JVM executes a single implementation shared by all generic instantiations. A `Container<String>` and a `Container<Integer>` therefore use the same erased runtime representation.

C++ templates follow a different execution model. Template instantiation occurs during compilation, and the compiler generates a specialized version of the code for every concrete type used by the program. For example:

```
template <typename T>
class Container {

    T value;

public:

    T get() {
        return value;
    }
};
```

instantiated as:

```
Container<int> a;
Container<double> b;
```

produces two independent implementations specialized for `int` and `double`. This process, known as monomorphization, allows the compiler to exploit complete type information during optimization.

Therefore, Java preserves a uniform runtime representation by removing generic type information, while C++ exchanges additional generated code for type-specific machine implementations.

2.3.1 Memory Representation and Pointer Indirection

The main performance consequence of Java type erasure is that generic structures operate primarily on object references rather than direct values. In the JVM object model, reference variables contain pointers to heap objects, requiring additional memory accesses compared with direct value storage.

For example:

```
ArrayList<Integer> values = new ArrayList<>();

values.add(10);

int x = values.get(0);
```

does not store the integer value directly inside the container. Instead, the runtime representation consists of a reference to an `Integer` object:

$$\text{Array reference} \rightarrow \text{Object reference} \rightarrow \text{Heap object containing value}$$

Accessing an element therefore requires additional pointer dereferencing and memory accesses.

In contrast, C++ templates combined with value semantics allow direct storage of objects inside contiguous memory regions. For example:

```
std::vector<int> values;
```

stores integer values directly:

$$\text{int} \rightarrow \text{int} \rightarrow \text{int} \rightarrow \text{int}$$

The absence of intermediate references reduces memory latency and improves the ability of the processor to exploit cache locality (a systematic comparison with C++ is given in Section 2.3.6).

2.3.2 Boxing and Unboxing Overhead

A significant limitation of Java Generics is the inability to directly instantiate generic types with primitive values. Generic parameters must always refer to object types; therefore primitive values require wrapper objects.

For example:

```
ArrayList<Integer> values = new ArrayList<>();

values.add(5);
```

requires automatic boxing:

$$int \rightarrow Integer$$

while retrieving the value requires unboxing:

$$Integer \rightarrow int$$

The boxing process introduces additional execution costs:

- creation of wrapper objects when values are not reused;
- additional memory accesses caused by object references;
- increased memory footprint due to object metadata;
- conversion operations during boxing and unboxing.

These overheads are particularly relevant for numerical workloads, where large collections of primitive values are processed repeatedly (C++ templates avoid this limitation entirely, as discussed in Section 2.3.6).

2.3.3 Runtime Effects of Type Erasure: Casts and Bridge Methods

Type erasure also requires Java to preserve type safety through inserted casts. Since the JVM executes erased bytecode, the compiler introduces runtime checks when retrieving values from generic structures.

For example:

```
String value = list.get(0);
```

is internally equivalent to:

```
String value = (String) list.get(0);
```

The cast introduces an additional runtime verification step. Modern JVM implementations can frequently eliminate redundant checks through JIT optimization, but unpredictable execution paths may prevent complete removal.

Another consequence of type erasure is the generation of synthetic bridge methods. Consider:

```
interface Box<T> {

    T get();
```

```
}

class StringBox implements Box<String> {

    public String get() {
        return "value";
    }

}
```

After erasure, the interface method becomes:

```
Object get();
```

Therefore, the compiler generates an additional bridge method:

```
public Object get() {

    return get();

}
```

The additional invocation layer can introduce a small runtime overhead. However, HotSpot can often remove this cost through method inlining when the runtime profile provides sufficient information.

2.3.4 Cache Locality and Data Layout

The memory layout produced by generic structures strongly influences cache behavior.

A Java container containing reference types generally follows the layout:

$$\text{Container storage} \rightarrow \text{Reference} \rightarrow \text{Heap object} \rightarrow \text{Value}$$

Objects may be distributed across different regions of memory, reducing spatial locality. During sequential traversal, the processor may need to load multiple independent cache lines because consecutive logical elements are not necessarily physically adjacent.

This increases cache misses and reduces effective memory bandwidth, in contrast with the contiguous value storage achievable through C++ templates (Section 2.3.6).

2.3.5 JVM Just-In-Time Optimization Techniques

Although Java Generics introduce runtime overheads, modern JVM implementations attempt to reduce these costs through adaptive optimization.

The HotSpot JVM performs runtime profiling and dynamically generates optimized machine code for frequently executed paths. Relevant techniques include:

- **Devirtualization**: replacing dynamic method resolution with direct calls when the receiver type is predictable.

- **Inline caching**: recording observed runtime types at call sites to optimize repeated method invocations.
- **Escape analysis**: determining whether objects remain local to a method and enabling scalar replacement or elimination of unnecessary allocations.

For example, if an object created inside a method never escapes, the JVM may replace the object allocation with direct scalar values, reducing memory access overhead. However, these optimizations remain speculative and depend on runtime profiling, unlike the ahead-of-time specialization guaranteed by C++ templates.

2.3.6 Comparison with C++ Templates

The preceding subsections isolate individual runtime costs introduced by Java's type-erasure model; this section consolidates them into a single systematic comparison with C++ templates.

At the representation level, Java containers store object references, introducing pointer indirection and additional memory accesses, whereas C++ templates instantiated with concrete types allow direct, contiguous value storage. This layout difference directly affects cache behavior: Java's reference-based objects can be scattered across the heap, increasing cache misses during traversal, while C++ value layouts enable efficient cache-line utilization, hardware prefetching, and vectorization.

At the level of primitive handling, Java's inability to instantiate generics with primitive types forces boxing and unboxing whenever numeric values are stored in generic structures, adding allocation and conversion costs absent from C++, where a template instantiated with a primitive type manipulates its native representation directly.

At the level of code generation, the two models also diverge on instruction cache behavior. Java type erasure generates a single implementation shared among all generic instantiations, keeping executable size small but preventing the generated code from exploiting concrete type information. C++ monomorphization instead generates a separate implementation per instantiated type: this increases executable size and can introduce instruction-cache pressure ("code bloat"), but enables stronger optimizations such as complete function inlining, constant propagation, and architecture-specific code generation.

Finally, the two models differ in *when* optimization decisions are made. C++ performs optimization during ahead-of-time compilation, with the compiler having complete information about data representation, memory layout, and available operations before the program ever runs. Java follows a dynamic model instead, in which the JVM profiles the running program and generates JIT-optimized machine code based on observed behavior. Consequently, C++ provides predictable, guaranteed specialization, while Java provides adaptive optimization capable of exploiting information that is only available at runtime.

Characteristic	Java Generics	C++ Templates
Generic representation	Type erasure with shared runtime representation	Compile-time monomorphization
Primitive handling	Boxing and unboxing required	Direct native representation
Memory access	Reference indirection and pointer chasing	Direct contiguous storage
Dispatch	Potential runtime dispatch	Static compile-time dispatch
Optimization	Adaptive JIT optimization	AOT compiler optimization
Cache locality	Reduced by object-based layouts	Improved through value layouts

Table 2.1: Runtime performance comparison between Java Generics and C++ templates.

2.3.7 Comparative Performance Summary

2.4 Type Safety

Type safety is a fundamental property of statically typed programming languages and defines the ability of a type system to prevent invalid operations on values before program execution. In the context of parametric polymorphism, type safety requires that abstract type parameters can be substituted with concrete types without introducing operations that violate the constraints established by the program.

A sound type system guarantees that every expression accepted by the compiler respects the typing rules of the language. This property is traditionally described through two fundamental principles:

- *Progress*: a well-typed program cannot reach a state where an undefined operation caused by a type mismatch must be performed;
- *Preservation*: the evaluation of a well-typed expression preserves its type correctness.

Parametric polymorphism extends these guarantees by allowing the definition of algorithms and data structures independently from specific types. However, the implementation strategy adopted by a language determines how type information is preserved during compilation and how the type system guarantees correctness.

The implementation strategy adopted by Java and C++ differs in how type information is preserved after compilation; this distinction is examined in detail in Section 2.4.5, once the static type checking rules of Java Generics have been introduced.

2.4.1 Static Type Checking in Java Generics

Java Generics extend the Java type system by introducing parameterized types. A generic declaration defines a family of related types whose correctness depends on

the constraints imposed on their type parameters.

Consider the following generic class:

```
class Box<T> {

    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

The type parameter `T` represents an abstract type that is determined when the generic class is instantiated.

For example:

```
Box<String> textBox = new Box<>();

textBox.set("Java");

String value = textBox.get();
```

The compiler interprets `Box<String>` as a type where every use of `T` must be compatible with `String`. Therefore, the following operation is rejected:

```
textBox.set(10);
```

because the argument type `Integer` does not satisfy the type constraint established by the parameterization.

The type system therefore prevents the creation of inconsistent states: after the declaration of `Box<String>`, every value inserted into the container must conform to the declared type argument.

The same principle applies to method invocations. If a generic method requires a specific type relationship, the compiler verifies that all actual arguments satisfy the required constraints before accepting the program.

For example:

```
static <T> T identity(T value) {

    return value;
}


String result = identity("example");
```

The compiler infers the type argument and ensures that the inferred substitution preserves the validity of the expression.

2.4.2 Bounded Type Parameters and Generic Constraints

Java generics also support bounded type parameters, allowing the type system to restrict the set of admissible substitutions.

For example:

```
class Maximum<T extends Comparable<T>> {

    T max(T a, T b) {

        return a.compareTo(b) > 0 ? a : b;
    }
}
```

The declaration:

```
<T extends Comparable<T>>
```

introduces a constraint stating that every valid type argument must provide a comparison operation compatible with the generic algorithm.

The compiler verifies this constraint when the generic type is instantiated. A type that does not satisfy the required relationship is rejected statically.

Therefore, Java generic constraints express properties that must hold for every valid substitution of the type parameter, allowing the type system to reason about generic programs independently from concrete types.

2.4.3 Type Erasure and Preservation of Static Type Safety

Java implements generics through type erasure. From the perspective of the type system, this means that generic parameters are used during static analysis but are not represented as independent types after the compilation process.

The important property is that erasure does not remove the guarantees provided by the type system. Generic correctness is established before the erasure transformation occurs.

For example:

```
List<String> names = new ArrayList<>();

names.add("Alice");

String name = names.get(0);
```

The compiler verifies that:

- only values compatible with `String` can be inserted into the list;
- every retrieval operation is interpreted as producing a value of type `String`.

After the generic verification phase, the implementation no longer depends on the explicit parameter `String`. However, the soundness argument of the type system

is based on the fact that all possible uses of the generic abstraction have already been checked.

Consequently, Java separates the concepts of:

- *static type correctness*, which is guaranteed by generic type checking;
- *type representation*, which is determined after the generic analysis phase.

The type-erasure strategy therefore preserves type safety by shifting the responsibility for correctness entirely to compile-time verification.

2.4.4 Reification-Based Type Safety in C++ Templates

C++ templates adopt a different approach based on type reification. Instead of removing type parameters after checking, the compiler instantiates templates using concrete type arguments.

Consider:

```
template <typename T>
class Box {

private:

    T value;

public:

    void set(T value) {
        this->value = value;
    }

    T get() {
        return value;
    }
};
```

A concrete use:

```
Box<int> integerBox;

integerBox.set(10);

int value = integerBox.get();
```

causes the compiler to analyse the template using the concrete type `int`. Every operation involving `T` is therefore checked against the actual instantiated type.

If an invalid operation is attempted:

```
Box<int> box;

box.set("text");
```

the compiler rejects the program because the instantiated type `int` does not accept a string value.

From the perspective of type safety, C++ templates preserve the complete type information during template instantiation. The compiler verifies the correctness of each concrete specialization directly rather than checking an abstract erased representation.

2.4.5 Comparison of Java Generics and C++ Templates

Java Generics and C++ Templates both provide compile-time guarantees through parametric polymorphism, but they preserve type information in different ways.

Java follows the model:

```
Generic abstraction
        |
        v
Static type checking
        |
        v
Type erasure
```

The type system verifies every substitution before removing the generic parameter from the resulting representation.

C++ follows the model:

```
Template abstraction
        |
        v
Instantiation with concrete type
        |
        v
Static type checking of specialization
```

The compiler retains the concrete type throughout template processing and verifies operations directly on the instantiated form.

Therefore, the fundamental distinction is not whether type safety is provided, but how the type system associates abstract parameters with concrete types. Java achieves safety through static verification before erasure, whereas C++ achieves safety through compile-time reification of concrete types.

2.5 Reflection and Runtime Type Information

The implementation of parametric polymorphism in Java is fundamentally based on *type erasure*. Unlike languages adopting a reified representation of generic types, the Java compiler translates every parameterized type into a non-generic representation before bytecode generation. During compilation, each type parameter is replaced by

its upper bound if one is declared, or by `java.lang.Object` otherwise. In addition, the compiler automatically inserts the necessary type casts to preserve static type safety and generates bridge methods when required to maintain binary compatibility under inheritance and method overriding.

Consequently, generic type parameters exist exclusively within the static type system enforced by the compiler. The generated JVM bytecode contains no runtime distinction between different instantiations of the same generic declaration. For example, the declarations `List<String>` and `List<Integer>` are both translated into the same runtime representation corresponding to the raw type `java.util.List`. The Java Virtual Machine therefore executes a single runtime implementation regardless of the concrete type arguments that were present in the source program.

This compilation strategy implies that parameterized type arguments are not represented as independent runtime entities. The JVM preserves metadata describing the generic declaration itself through the class file's *Signature* attribute, allowing reflective APIs to reconstruct generic declarations under specific circumstances. However, the runtime representation of ordinary object instances contains only the erased type, meaning that dynamically allocated objects do not retain the concrete type arguments with which they were instantiated.

2.5.1 Reflection and RTTI Limitations

Java's Runtime Type Information (RTTI) mechanism is centered on the `java.lang.Class` object associated with every loaded class. Since all parameterized instantiations of a generic class are erased to the same runtime type, reflection operates exclusively on the erased representation rather than on the original parameterized form.

A direct consequence is that parameterized generic types cannot participate in runtime type tests. The Java Language Specification explicitly prohibits expressions such as

```
obj instanceof List<String>
```

because the runtime system has no information capable of distinguishing `List<String>` from `List<Integer>`. Allowing such expressions would incorrectly suggest that the JVM could recover type arguments that no longer exist after compilation.

Similarly, the `getClass()` method always returns the runtime class corresponding to the erased type. Two objects instantiated with different generic arguments therefore produce identical runtime class descriptors.

```
import java.util.*;

public class ReflectionExample {

    public static void main(String[] args) {

        List<String> strings = new ArrayList<>();
        List<Integer> integers = new ArrayList<>();

        System.out.println(strings.getClass());
```

```
        System.out.println(integers.getClass());

        System.out.println(strings.getClass()
                == integers.getClass());

        // Compilation error:
        // if (strings instanceof List<String>) { }
    }

}
```

The program produces equivalent runtime class objects for both lists, demonstrating that the generic arguments have been removed during compilation. The comparison evaluates to `true` because both variables reference instances of the same erased runtime class.

Reflection APIs such as `Class`, `Method`, and `Field` therefore identify only erased runtime types during ordinary object inspection. Although generic declarations remain partially accessible through reflective interfaces such as `java.lang.reflect.Type` and `ParameterizedType`, this information originates from generic signatures stored in class metadata rather than from the runtime representation of individual object instances. Consequently, reflection cannot determine the concrete type argument associated with an arbitrary generic object unless that information has been preserved explicitly by the programmer.

2.5.2 Type Tokens as an Explicit Runtime Representation

Because type erasure removes concrete generic arguments from ordinary runtime objects, Java libraries frequently employ *type tokens* to preserve type information explicitly. A type token consists of passing a `Class<T>` object alongside the generic value, thereby making the runtime type available independently of the erased generic parameter.

The following example illustrates this approach.

```
public class Box<T> {

    private final Class<T> type;
    private final T value;

    public Box(Class<T> type, T value) {
        this.type = type;
        this.value = value;
    }

    public Class<T> getType() {
        return type;
    }

    public T getValue() {
        return value;
```

```
    }

    public static void main(String[] args) {

        Box<String> box = new Box<>(String.class, "Hello");

        System.out.println(box.getType().getName());

        if (box.getType() == String.class) {
            System.out.println("String detected.");
        }
    }

}
```

The runtime type information in this example is not recovered from the generic parameter itself but from the explicitly supplied `Class<String>` object. This pattern is extensively employed by serialization frameworks, dependency injection containers, persistence libraries, and reflection-based infrastructures that require runtime knowledge of application types despite Java's erased implementation of generics.

2.5.3 Comparison with C++ Template RTTI

The reflection semantics of Java differ fundamentally from those of C++ because the two languages adopt distinct implementation strategies for parametric polymorphism.

C++ templates are implemented through *monomorphization*. During compilation, every template specialization generates an independent concrete type with its own layout, member functions, and associated runtime type metadata. Consequently, specializations such as `std::vector<int>` and `std::vector<double>` constitute distinct runtime types rather than different views of a shared implementation.

When Run-Time Type Information is enabled, the C++ runtime associates each polymorphic specialization with its own RTTI structures. Facilities such as `typeid` and `dynamic_cast` therefore operate on specialized concrete types generated by template instantiation. Each specialization possesses an independent `std::type_info` object corresponding to its unique runtime identity.

Java follows the opposite approach. All parameterized instantiations of a generic declaration are translated into a single erased runtime class represented by one shared `java.lang.Class` object. Reflection therefore exposes the identity of the generic declaration after erasure rather than the original parameterized instantiation appearing in the source code.

The distinction is summarized as follows:

- Java implements parametric polymorphism through type erasure, causing all parameterized instantiations of a generic class to share a single runtime type.

- Java reflection and RTTI operate exclusively on erased runtime classes, preventing direct runtime discrimination between different generic arguments.
- C++ implements templates through monomorphization, generating distinct concrete types for every specialization.
- C++ RTTI associates independent runtime metadata with each specialization, enabling facilities such as `typeid` and `dynamic_cast` to distinguish specialized template types directly.

These contrasting implementation strategies illustrate the fundamental trade-off between preserving runtime type identity through reification, as in C++, and reducing runtime type diversity through type erasure, as adopted by the Java Virtual Machine.

Chapter 3

C# Generics

3.1 Instantiation Time

C# generics adopt a runtime-oriented instantiation model that differs fundamentally from both the compile-time monomorphization strategy of C++ templates and the type erasure strategy of Java generics. The defining characteristic of the C# approach is that generic type definitions are compiled before execution, while the generation of their concrete executable representation is delayed until runtime by the Common Language Runtime (CLR) and its Just-In-Time (JIT) compiler.

In C#, generic declarations are not immediately expanded into specialized versions during compilation. Instead, the C# compiler translates generic classes and methods into Common Intermediate Language (CIL), preserving generic parameters as metadata entities. The resulting assembly contains a generic type definition together with information about its type parameters and constraints, but it does not necessarily contain native code for each possible generic instantiation.

For example, the generic definition:

```
class Container<T>
{
    public T value;
}
```

is compiled once into a generic CLI type definition. The compiler does not generate separate implementations such as:

```
Container<int>
Container<double>
Container<string>
```

at compilation time. The concrete instantiation is postponed until execution.

3.1.1 CLR and JIT Generic Instantiation Mechanism

The instantiation process begins when the CLR loads an assembly containing a generic definition. At this stage, the runtime knows the existence of the generic type but does not necessarily generate machine code for any concrete specialization.

A concrete generic instantiation is created when program execution requires a specific constructed type. For example, when the runtime encounters:

```
Container<int> c;
```

the CLR resolves the generic definition `Container<T>` together with the type argument `int`. This produces a runtime representation of the constructed generic type `Container<int>`.

Only when executable code for this instantiation is required does the JIT compiler translate the corresponding CIL into native machine instructions. Therefore, the timeline of C# generic instantiation can be summarized as:

1. The C# compiler generates a generic CIL definition.
2. The assembly is loaded by the CLR.
3. A concrete generic instantiation is requested during execution.
4. The CLR creates the constructed generic type.
5. The JIT compiler generates native code when required.

Consequently, C# generics are instantiated at runtime rather than during source compilation.

The CLR does not necessarily create a completely independent native implementation for every generic instantiation. Instead, the JIT compiler applies different strategies depending on the category of the type argument.

For reference types, the CLR can usually share the same generated native code between different instantiations because all reference types have compatible managed reference representations. Therefore, instantiations such as:

```
List<string>
List<object>
List<MyClass>
```

may reuse a single JIT-generated implementation.

For value types, the situation is different. Value types have different memory layouts and sizes, meaning that a single shared implementation cannot always be used. Therefore, the CLR JIT compiler generates specialized native code for different value-type instantiations:

```
List<int>
List<double>
```

represent separate JIT specialization events.

Thus, the C# instantiation model is hybrid: runtime instantiation occurs in both cases, but reference types generally use runtime code sharing, whereas value types trigger runtime specialization.

3.1.2 Comparison with C++ Templates: Runtime Instantiation vs Compile-Time Instantiation

The C# runtime instantiation model differs from the C++ template model primarily because the moment at which specialization occurs is different.

C++ templates use compile-time instantiation, commonly described as monomorphization. When the compiler encounters:

```
Vector<int>
Vector<double>
```

it generates separate concrete implementations during compilation. The executable produced by the compiler already contains the specialized versions before execution.

The timeline is therefore:

Template Definition → Compile-Time Instantiation → Native Code Generation → Execution

In contrast, the C# timeline is:

Generic Definition → CIL Generation → Runtime Instantiation → JIT Compilation → Execution

The essential difference is that C++ performs specialization ahead of execution, whereas C# delays specialization until the runtime environment determines that a specific generic instantiation is required.

3.1.3 Comparison with Java Generics: Runtime Instantiation vs Type Erasure

Java generics follow a fundamentally different model based on type erasure. During Java compilation, generic parameters are removed and replaced by their erasures. Consequently, the Java Virtual Machine does not instantiate generic types at runtime.

For example:

```
List<String>
List<Integer>
```

are compiled into the same erased representation:

```
List
```

The JVM executes a single non-specialized implementation, and no runtime generic instantiation event occurs.

The timeline of Java generics is therefore:

Generic Source Code → Compile-Time Erasure → Bytecode Generation → Execution

This differs from C#, where the generic definition survives compilation and concrete instantiation is performed during execution.

Therefore, the three models can be summarized as:

The defining property of C# generics is therefore not merely the preservation of generic information, but the placement of the instantiation event: generic types are materialized by the runtime system when execution requires them, with the JIT compiler acting as the specialization mechanism.

Language	Instantiation Time	Specialization Phase
C++	Compile time	Compiler monomorphization
C#	Runtime	CLR/JIT instantiation
Java	Compile time	Type erasure, no specialization

3.2 Runtime Representation

C# generics provide a runtime implementation of parametric polymorphism based on *reification*. In a reified generic system, information about generic type parameters is preserved after compilation and remains available to the execution environment.

From a theoretical perspective, parametric polymorphism allows a program component to operate independently from the concrete types on which it is instantiated. A generic definition can be represented as:

$$G\langle T\rangle$$

where T is a type variable that can later be replaced by a concrete type argument. However, the concept of parametric polymorphism does not define how generic abstractions are represented at runtime. The implementation requires decisions about metadata representation, code generation, object layout, and execution mechanisms.

The CLR implements generics through a hybrid model that combines runtime reification with selective specialization. Generic definitions are preserved inside compiled assemblies, while the final executable representation is generated dynamically by the Just-In-Time (JIT) compiler.

Conceptually, the C# compilation pipeline can be represented as:

C# Source → CIL + Metadata → CLR Generic Instantiation → JIT Compilation → Native Code

This architecture allows the runtime to preserve generic type identity while deciding dynamically whether a particular instantiation requires a specialized implementation or whether an existing implementation can be shared.

3.2.1 CLR Execution Engine and CIL Metadata Representation

The first stage of the CLR generic implementation occurs during compilation. The C# compiler translates source code into *Common Intermediate Language* (CIL) and generates metadata according to the Common Language Infrastructure (CLI) specification defined by ECMA-335.

Unlike Java bytecode after type erasure, the generated assembly preserves explicit information about generic declarations. A generic type is represented as an open generic definition containing formal type parameters.

For example:

```
List<T>
```

is stored as a generic type definition rather than as a concrete implementation.

The CLR metadata system contains dedicated structures for representing generic relationships. The most relevant metadata tables include:

- `GenericParam`, which stores information about generic parameters;
- `GenericParamConstraint`, which stores constraints associated with generic parameters;
- `TypeSpec`, which represents constructed types;
- `MethodSpec`, which represents constructed generic methods.

These structures allow the CLR loader and execution engine to reconstruct the relationship between a generic definition and its concrete arguments.

For example, the runtime can represent:

$$\texttt{List<T>} \rightarrow \texttt{List<int>}$$

as an explicit construction relationship, preserved as part of the runtime type system rather than resolved away as in a type-erasure model.

3.2.2 Runtime Instantiation and JIT Compilation

A fundamental characteristic of CLR generics is that instantiation occurs at runtime rather than during the initial compilation phase. The compiler emits generic definitions, while the CLR creates concrete representations when a closed constructed type is required.

A generic definition:

$$G\langle T\rangle$$

becomes a runtime constructed type:

$$G\langle U\rangle$$

where U is a concrete type argument.

For example:

```
Dictionary<TKey,TValue>
```

may generate runtime instantiations such as:

```
Dictionary<int,string>
```

or:

```
Dictionary<Guid,Object>
```

The CLR execution engine resolves these constructed types and provides the necessary information to the JIT compiler. The JIT compiler then generates native machine code according to the characteristics of the supplied type arguments, performing this instantiation step as part of runtime execution rather than during compilation.

3.2.3 Value-Type Specialization

When a generic type is instantiated with a value type, the CLR generally produces a specialized native implementation for that concrete type. Value types such as `int`, `double`, or user-defined structures have their own memory layouts and representation requirements. Consequently, sharing a single machine-code implementation between unrelated value types would require additional abstraction mechanisms.

For example:

```
List<int>
```

and:

```
List<double>
```

require different representations of their stored elements. Therefore, the JIT compiler generates specialized code:

$$JIT(\texttt{List<int>}) = NativeCode_{int}$$

and:

$$JIT(\texttt{List<double>}) = NativeCode_{double}$$

The generated implementations contain concrete type information and operate directly on the corresponding value representation, produced by the CLR at runtime rather than by the compiler ahead of execution.

3.2.4 Reference-Type Code Sharing

Generic instantiations based on reference types follow a different execution strategy. Because all reference types share a common object-reference representation inside the CLR, the runtime can reuse the same native implementation for multiple generic instantiations.

For example:

```
List<string>
```

and:

```
List<object>
```

may share the same generated machine code.

Conceptually, instead of producing:

$$JIT(\texttt{List<string>}) = NativeCode_{string}$$

and:

$$JIT(\texttt{List<object>}) = NativeCode_{object}$$

the CLR can generate a shared implementation:

$$NativeCode_{reference}$$

where runtime-specific information is supplied through additional execution-engine structures.

This mechanism provides a balance between specialization and code reuse: the CLR avoids generating an independent implementation for every reference-type instantiation, while still retaining the exact generic type arguments needed to preserve type safety at runtime.

3.2.5 Runtime Dictionaries and Method Tables

To support shared generic implementations, the CLR uses runtime data structures that provide information about concrete generic arguments. These structures are commonly described as generic dictionaries or runtime generic contexts.

When shared machine code executes, operations requiring type-specific information can access these runtime structures to resolve:

- type descriptors;
- method addresses;
- field layouts;
- interface implementations.

This mechanism is conceptually related to dictionary passing techniques used in implementations of parametric polymorphism. The generated code remains generic, while additional runtime information supplies the missing concrete type details – a mechanism that has no equivalent in either C++ template instantiation, where all parameters are resolved before execution, or Java's type-erasure model, where generic parameters are removed before bytecode execution rather than resolved at run time.

3.2.6 Runtime Type Identity

A defining property of CLR reification is that constructed generic types maintain their runtime identity. The CLR considers different generic instantiations as distinct runtime types.

Therefore:

$$RuntimeType(\texttt{List<int>}) \neq RuntimeType(\texttt{List<string>})$$

This capability is fundamentally different from Java, where:

$$RuntimeType(\texttt{List<String>}) = RuntimeType(\texttt{List<Integer>}) = \texttt{List}$$

after erasure.

C++ occupies a different position. Concrete template specializations may participate in RTTI if they correspond to runtime types, but the original template definition is not preserved as a runtime entity. The executable contains information about concrete types rather than about the generic abstraction that produced them. The reflection system built on top of this runtime type identity is discussed in Section 3.5.

3.2.7 Architectural Comparison

The runtime representation strategies can be summarized as follows:

Feature	C# Generics	Java Generics	C++ Templates
Generic model	Reification	Type Erasure	Monomorphization
Metadata presence	Preserved in CLR	Removed from JVM execution	Not available
Instantiation time	Runtime/JIT	Compile-time only	Compile-time
Code generation	Specialization and sharing	Single erased bytecode	Native specialization
Runtime identity	Generic arguments preserved	Generic arguments removed	Concrete types only
Generic runtime object	Yes	No	No

Table 3.1: Comparison of generic runtime representation models.

3.3 Performance Characteristics

C# generics adopt a runtime reification model implemented by the .NET Common Language Runtime (CLR). Unlike Java Generics, where generic information is removed through type erasure, C# generic type parameters are preserved inside the Common Intermediate Language (CIL) metadata and remain available during execution. This allows the runtime environment to distinguish different generic instantiations and apply different execution strategies depending on the concrete type.

The CLR combines generic reification with Just-In-Time (JIT) compilation. Generic classes and methods are initially compiled into a type-independent CIL representation. When a concrete instantiation is required, the JIT compiler generates native machine code according to the characteristics of the involved type. Therefore, C# generics do not correspond to a purely compile-time expansion mechanism as in C++ templates, but they are also not completely erased as in Java.

The CLR execution strategy depends mainly on whether the generic parameter is a value type or a reference type:

$$\text{Generic<T> execution} = \begin{cases} \text{specialized native code} & \text{if } T \text{ is a value type} \\ \text{shared native code} & \text{if } T \text{ is a reference type} \end{cases}$$

For value types such as `int`, `float`, or user-defined structures, the JIT compiler generates specialized implementations because different value types have different sizes, alignment requirements, and calling conventions. This allows direct manipulation of the actual data representation without additional indirection.

For reference types, instead, the CLR can share the same native implementation between different instantiations because object references have a uniform runtime representation. The execution engine maintains the required runtime type information through internal generic context structures, avoiding the need to generate duplicated machine code for every reference type.

This hybrid model provides a compromise between the complete specialization of C++ templates and the erased execution model of Java Generics.

3.3.1 Value-Type Instantiations: Memory Layout and Cache Efficiency

The main performance advantage of C# generics appears when generic data structures are instantiated with value types. In this case, the elements are stored directly inside the containing structure, avoiding object allocation and reference indirection.

Consider the following example:

```
public struct Vector3
{
    public float X;
    public float Y;
    public float Z;
}

public class Buffer<T>
{
    private T[] data;

    public Buffer(int size)
    {
        data = new T[size];
    }

    public float Compute()
    {
        float result = 0;

        for(int i = 0; i < data.Length; i++)
        {
            result += data[i].GetHashCode();
```

```
        }

        return result;
    }
}
```

When instantiated with a value type:

```
Buffer<Vector3> points =
    new Buffer<Vector3>(1000000);
```

the array contains the actual structures:

(x_0, y_0, z_0)	(x_1, y_1, z_1)	(x_2, y_2, z_2)

The memory representation is contiguous, allowing efficient sequential access. Because each element is stored directly inside the array, the processor can exploit spatial locality and efficiently use cache lines.

The execution cost is approximately:

$$T_{value} = N(C_{load} + C_{operation})$$

where the memory access component is minimized by predictable sequential loads.

Furthermore, no boxing operation is required. Primitive values and structures remain in their native representation, avoiding the additional memory accesses required by object wrappers. This makes C# generic collections of value types closer to the execution model of specialized C++ template instantiations.

3.3.2 Reference-Type Instantiations: Indirection and Shared Code

When a generic parameter is a reference type, the CLR uses code sharing. For example:

```
Buffer<VectorClass> objects =
    new Buffer<VectorClass>(1000000);
```

where:

```
public class VectorClass
{
    public float X;
    public float Y;
    public float Z;
}
```

the array stores references rather than the objects themselves:

ptr_0	ptr_1	ptr_2

The actual objects are located elsewhere in memory. Accessing an element requires an additional pointer dereference:

$$T_{reference} = N(C_{load\ reference} + C_{pointer\ chase} + C_{operation})$$

The additional memory access can reduce cache efficiency because the referenced objects may not be stored contiguously. Consequently, large collections of reference types generally exhibit weaker spatial locality compared with value-type collections.

However, unlike Java erased generics, this behavior does not require boxing of primitive values because C# maintains explicit runtime support for generic types.

3.3.3 Comparison with C++ Template Monomorphization

C++ templates use compile-time monomorphization. For every concrete template instantiation, the compiler generates a separate specialized implementation before program execution.

Example:

```
template<typename T>
class Buffer
{
    T* data;

public:

    T sum()
    {
        T result = 0;

        for(int i = 0; i < size; i++)
            result += data[i];

        return result;
    }
};

Buffer<float> a;
Buffer<double> b;
```

The compiler produces two independent versions:

$$Buffer < float > \rightarrow Code_{float}$$

$$Buffer < double > \rightarrow Code_{double}$$

This approach provides maximum optimization opportunities because the compiler has complete knowledge of the concrete type. The generated code can exploit static dispatch, aggressive inlining, and type-specific optimizations.

Compared with C#, C++ templates have the advantage that specialization happens before execution. However, every instantiation generates additional machine code, potentially increasing executable size and instruction-cache pressure.

C# generics avoid this duplication for reference types while still providing specialized execution for value types through JIT compilation.

3.3.4 Comparison with Java Type-Erased Generics

Java Generics follow a fundamentally different execution model based on type erasure. Generic parameters are removed during compilation and replaced with their upper bounds in the generated bytecode. A generic class:

```
class Buffer<T>
{
    private T[] data;

    T get(int index)
    {
        return data[index];
    }
}
```

is represented at runtime without knowledge of the original type parameter.

Because the JVM does not generate specialized versions for primitive types, generic collections require wrapper objects:

$$int \rightarrow Integer \rightarrow reference$$

For example:

```
ArrayList<Integer> values =
    new ArrayList<Integer>();
```

stores references to objects rather than compact primitive values. Accessing an element requires additional indirection and possible boxing/unboxing operations.

The resulting execution model introduces:

$$T_{Java} = T_{operation} + C_{boxing} + C_{indirection}$$

which can negatively affect memory locality and cache efficiency.

3.3.5 Summary of Runtime Performance Characteristics

The three approaches provide different trade-offs:

	C++ Templates	C# Generics	Java Generics
$Genericrepresentation$	$Compile-timespecialization$	$Runtimereification$	$Typeerasure$
$Valuetypestorage$	$Direct$	$Direct$	$Usuallyboxed$
$Codegeneration$	$AOTmonomorphization$	$JITspecialization$	$Sharederasedco$
$Cachelocality$	$Veryhigh$	$Highforvaluetypes$	$Lowerduetoindirec$
$Runtimeoverhead$	$Minimal$	Low	$Higherforprimiti$

C# generics therefore represent an intermediate solution between C++ and Java. They preserve runtime type information and allow efficient specialized execution for value types, while avoiding excessive code duplication through reference type sharing. In terms of execution performance, C# approaches the efficiency of C++ templates for value-oriented workloads while providing a more flexible runtime model than Java erased generics.

3.4 Type Safety

Type safety represents one of the fundamental properties of a sound type system. A type-safe language guarantees that well-typed programs cannot perform operations that violate the rules defined by the language type model. In the context of parametric polymorphism, type safety requires that generic abstractions preserve the relationships between type parameters and concrete types throughout the program.

From a theoretical perspective, a sound type system is generally described through the properties of *progress* and *preservation*. The preservation property states that the evaluation of a well-typed expression cannot invalidate its typing judgment:

$$\Gamma \vdash e : T \land e \to e' \implies \Gamma \vdash e' : T$$

where Γ represents the typing environment, e is an expression, and T is the type assigned by the type system.

The progress property states that a well-typed expression is either already a value or can perform a valid computation step:

$$\Gamma \vdash e : T \implies e \text{ is a value or } e \to e'$$

For generic programming, these principles imply that a generic abstraction must remain type-correct for every valid substitution of its type parameters. A generic definition cannot assume properties that are not guaranteed by the type system, and every operation performed on a type parameter must be justified by the available typing information.

C# generics implement parametric polymorphism while preserving these properties through static verification of generic type usage. The compiler verifies that all instantiations of a generic abstraction respect the constraints imposed by the generic declaration, preventing invalid substitutions from being accepted.

3.4.1 Static Type Checking of Generic Parameters

A generic type introduces a type variable whose concrete type is unknown during the declaration of the abstraction. However, this unknown type is not arbitrary: the type system reasons about it symbolically and ensures that every operation performed on the parameter is valid for all permitted substitutions.

Consider the following generic container:

```
public class Box<T>
{
    private T value;

    public void Set(T value)
    {
        this.value = value;
    }

    public T Get()
    {
        return value;
    }
}
```

The type parameter T represents an abstract type that will be replaced by a concrete type during generic instantiation. The type system guarantees that all values stored inside the container are compatible with the chosen instantiation.

For example:

```
Box<int> numbers = new Box<int>();

numbers.Set(10);

int value = numbers.Get();
```

The compiler verifies that the argument passed to `Set` is compatible with the type parameter substitution:

```
Box<int> numbers = new Box<int>();

numbers.Set("text"); // Type error
```

The assignment is rejected because the generic instance `Box<int>` defines a type invariant requiring every stored value to be an integer. The generic mechanism therefore does not weaken static typing but extends it by allowing reusable abstractions while preserving exact type relationships.

3.4.2 Generic Constraints as Type System Rules

In a generic declaration, a type parameter does not automatically provide access to arbitrary operations. The type system only permits operations that are valid for every possible type satisfying the current assumptions about the parameter.

For example, the following generic method cannot assume that T provides a specific member:

```
public class Processor<T>
{
    public void Process(T value)
    {
        value.SpecialOperation();
    }
}
```

The compiler rejects this definition because the generic parameter does not guarantee the existence of `SpecialOperation`. The operation would violate type safety for substitutions where such a member does not exist.

Generic constraints extend the typing environment by introducing additional assumptions about the parameter:

```
public class Processor<T>
    where T : IDisposable
{
    public void Process(T value)
    {
        value.Dispose();
    }
}
```

The constraint establishes that every valid substitution of T must implement `IDisposable`. Therefore, the invocation of `Dispose()` is statically verified as safe.

Constraints can be interpreted as additional typing rules restricting the set of types that can instantiate a generic abstraction. They do not represent runtime checks added after compilation, but rather static guarantees used by the type checker to prove correctness.

3.4.3 Variance and Type-Safe Generic Subtyping

Generic type safety also depends on the interaction between generics and subtyping. A naive interpretation of subtyping could incorrectly allow unsafe substitutions between parameterized types.

For example, although `string` is a subtype of `object`, a mutable generic container containing strings cannot generally be considered a container of objects:

```
List<string> strings = new List<string>();

List<object> objects = strings; // Invalid
```

Allowing this assignment would violate type safety because an object of another type could be inserted into the list through the `List<object>` reference. The generic type system therefore treats mutable generic classes as invariant.

C# explicitly supports variance annotations for generic interfaces and delegates when the substitution is type-safe. For example:

```
IEnumerable<string> strings =
    new List<string>();

IEnumerable<object> objects = strings;
```

This assignment is valid because `IEnumerable<T>` only provides read-only access to elements. Since values are produced but never consumed through the generic interface, replacing a more specific type with a more general one cannot introduce an invalid operation.

Covariance and contravariance therefore represent controlled extensions of the subtyping relation, where the type system permits only substitutions that preserve soundness.

3.4.4 Reified Generics and Preservation of Type Information

C# generics are based on a reified model in which generic type parameters remain associated with the generic abstraction. From the perspective of type safety, this means that the relationship between a generic definition and its concrete type arguments is preserved.

A generic type declaration:

```
Box<T>
```

can be instantiated with different type arguments:

```
Box<int>
Box<string>
```

These represent different constructed types derived from the same generic definition. The type system maintains the distinction between these parameterizations, ensuring that operations performed through one constructed type cannot violate the invariants of another.

This approach differs from type-erasure systems, where generic parameters are removed after compilation. In Java, for example, generic correctness is verified statically before erasure. The compiler guarantees that all uses of a generic abstraction are valid and may introduce implicit casts where necessary to preserve the source-level typing semantics.

Conceptually:

```
List<String> values = new ArrayList<>();

String element = values.get(0);
```

is checked using the generic type information available during compilation, even though the executable representation does not preserve the original parameterized type.

C# follows a different type-system model: the association between the generic abstraction and its type parameter remains part of the program representation.

Consequently, the generic type relationship is preserved rather than reconstructed after type erasure.

3.4.5 Comparison with C++ Templates from a Type System Perspective

C++ templates and C# generics both support parametric programming, but they represent different approaches to expressing type relationships.

In C++:

```
template<typename T>
class Box
{
    T value;
};
```

the compiler verifies whether a concrete substitution satisfies the requirements of the template when the template is instantiated. The resulting type correctness depends on the validity of the substituted operations.

C# generics instead define a parameterized type inside the language type system. The generic abstraction itself has a type-level meaning, and constraints explicitly describe the requirements that valid substitutions must satisfy.

Therefore, both mechanisms prevent invalid type usage, but they express generic correctness through different models:

- C++ templates rely on compile-time template substitution and validation of generated concrete types.
- C# generics represent parameterized types directly in the language type system and preserve the relationship between generic definitions and type arguments.

3.4.6 Type Safety Guarantees of the C# Generic Type System

The C# generic system preserves type safety through a combination of static checking rules and a precise representation of parameterized types. The main guarantees provided by the type system are:

- Every generic instantiation must satisfy the declared constraints of its type parameters.
- Operations performed on generic parameters are accepted only when they are valid for every permitted substitution.
- Generic subtyping relationships are restricted through variance rules to prevent unsafe substitutions.

- Generic abstractions preserve the association between type parameters and concrete types, maintaining the consistency of typing relationships.
- Invalid conversions between incompatible parameterized types are rejected statically.

Consequently, C# generics provide parametric polymorphism without sacrificing the guarantees of a statically typed language. The generic mechanism extends the expressiveness of the type system while preserving the fundamental property that well-typed programs cannot perform operations outside the set allowed by their declared types.

3.5 Reflection and Runtime Type Information

Unlike languages based on type erasure, the Common Language Runtime (CLR) implements a reified model of generics, where generic type information is preserved as part of the runtime representation of types. During compilation, C# source code is translated into Common Intermediate Language (CIL) instructions together with a rich metadata representation stored inside the assembly. Generic declarations, constraints, and instantiation relationships are encoded explicitly in this metadata and remain available during program execution.

A generic type declaration compiled into CIL is represented through metadata tables that preserve the association between a generic type definition and its parameters. For example, the declaration:

```
class Container<T>
{
}
```

is not transformed into a non-generic runtime entity. Instead, the CLR maintains a generic type definition with an explicit parameter placeholder. When a concrete instantiation such as:

```
Container<int>
Container<string>
```

is requested, the runtime creates a closed constructed type whose metadata maintains the correspondence between the generic parameter and the concrete type argument.

This mechanism is commonly referred to as *generic reification*, because the abstract structure of the generic type remains represented in the runtime type system. Consequently, both open generic types, where type parameters are not yet bound, and closed constructed types, where every parameter has been substituted by a concrete runtime type, can be inspected through the reflection subsystem.

The CLR exposes this information through the `System.Type` abstraction. A runtime type object contains intrinsic identity information describing the represented entity, including whether the type is generic, whether it represents a generic definition or a constructed instance, and which runtime types are associated with its generic parameters.

For example, the following distinctions are maintained by the runtime:

- `typeof(List<>)` represents an open generic type definition.
- `typeof(List<int>)` represents a closed constructed generic type.
- `typeof(List<int>).GetGenericArguments()` returns the concrete runtime type argument associated with the instantiation.

Therefore, reflection over C# generics operates on actual runtime type entities rather than on erased compile-time abstractions.

3.5.1 Runtime Inspection of Generic Types Using System.Reflection

The following example demonstrates how the reflection API can dynamically inspect a generic object, verify whether its runtime type is generic, identify the generic type definition, and retrieve the concrete type arguments used for instantiation.

```
using System;
using System.Collections.Generic;

class ReflectionExample
{
    static void InspectType(object instance)
    {
        Type runtimeType = instance.GetType();

        Console.WriteLine("Runtime type: " + runtimeType);

        if (runtimeType.IsGenericType)
        {
            Console.WriteLine("The inspected type is generic.");

            Type genericDefinition =
                runtimeType.GetGenericTypeDefinition();

            Console.WriteLine(
                "Generic definition: " + genericDefinition);

            Type[] arguments =
                runtimeType.GetGenericArguments();

            Console.WriteLine("Generic arguments:");

            foreach (Type argument in arguments)
            {
                Console.WriteLine(argument.FullName);
            }
        }
        else
```

```
        {
            Console.WriteLine("The inspected type is not generic.");
        }
    }

    static void Main()
    {
        List<int> values = new List<int>();

        InspectType(values);
    }
}
```

In this example, the call to `GetType()` retrieves the CLR runtime type descriptor associated with the object instance. The method `IsGenericType` verifies whether the runtime entity corresponds to a generic type. If this condition is satisfied, `GetGenericTypeDefinition()` extracts the original generic declaration, while `GetGenericArguments()` retrieves the concrete runtime types used to construct the closed generic instance.

The result of the inspection is therefore equivalent to observing the following runtime relationship:

$$\texttt{List<int>} \rightarrow \texttt{List<>} + \texttt{int}$$

where both the generic declaration and the concrete type substitution remain available through reflection.

3.5.2 Comparison with C++ Templates and Java Generics

The reflection capabilities of C# generics differ fundamentally from the runtime models adopted by C++ templates and Java generics. The distinction concerns the availability of generic type information after compilation and the possibility of recovering concrete type parameters through runtime introspection.

C++ Templates: Compile-Time Monomorphization and Limited RTTI

C++ templates follow a compile-time monomorphization model. A template definition acts as a compile-time pattern from which the compiler generates specialized entities for each concrete instantiation. The relationship between a template parameter and its instantiated type is therefore resolved during compilation rather than represented as a generic runtime object.

For example:

```
template<typename T>
class Container
{
};

Container<int> a;
```

```
Container<double> b;
```

does not produce a runtime generic type equivalent to the CLR representation of `Container<T>`. Instead, the compiler materializes concrete types associated with the instantiated arguments. After compilation, the executable contains the resulting concrete classes but does not generally preserve a runtime representation of the original template abstraction.

The C++ Runtime Type Information (RTTI) mechanism provides limited type introspection through facilities such as `typeid` and `std::type_info`. RTTI can identify the dynamic type of polymorphic objects and expose implementation-defined type information, but it does not provide a reflection system capable of enumerating template parameters or reconstructing the template instantiation relationship.

Consequently, an expression such as:

```
typeid(Container<int>)
```

can identify the resulting concrete type, but standard C++ RTTI cannot query the template argument `int` as a runtime metadata element. The association between the template and its parameters exists primarily as a compile-time property.

Java Generics: Type Erasure and Residual Metadata

Java generics adopt a fundamentally different strategy based on type erasure. During compilation, generic type parameters are removed from the runtime representation and replaced according to erasure rules, typically by their upper bounds. The Java Virtual Machine therefore executes code where generic parameters are not represented as runtime type identities.

For example:

```
List<String> names = new ArrayList<>();
```

is compiled into a representation equivalent to a non-parameterized collection type from the perspective of runtime object identity. The JVM can identify that the object is an instance of `ArrayList`, but the concrete parameter `String` is not retained as part of the object's runtime type information.

Java reflection can inspect generic declarations when metadata has been preserved in class files through structures such as the `Signature` attribute. However, this information describes declarations rather than runtime object instantiations. For example, reflection can determine that a class was declared as:

```
class Box<T>
```

but it cannot directly recover the concrete argument used by an arbitrary runtime object:

```
Box<String>
```

because the instantiated parameter is not part of the runtime type identity.

Therefore, Java reflection operates on residual generic metadata associated with declarations, whereas C# reflection operates on reified runtime type entities.

3.5.3 Comparative Summary of Runtime Generic Reflection

Considering only runtime metadata availability and type introspection capabilities, the three approaches can be summarized as follows:

- **C# Generics**: generic parameters are reified by the CLR. Both open generic definitions and closed constructed types retain runtime metadata and can be inspected through `System.Reflection`.
- **C++ Templates**: template parameters are resolved through compile-time monomorphization. Standard RTTI identifies concrete runtime types but does not expose template parameter metadata.
- **Java Generics**: generic parameters are erased from runtime type identity. Reflection can access declaration-level generic information but cannot directly inspect concrete type arguments of runtime instances.

The CLR generic model therefore provides a reflection system in which generic type structure remains part of the runtime type universe, whereas C++ and Java represent opposite design points: compile-time expansion without generic runtime metadata in C++, and runtime abstraction through type erasure in Java.